CutileReduce: Memory-Efficient Backpropagation for Monoidal Fold Kernels

Amaru Cuba Gyllensten

Abstract

Memory-efficient neural network kernels often avoid materializing large logical interaction tensors, but their backward passes are usually hand-derived. We identify a local-gradient condition for monoidal folds that permits backward computation from saved fold outputs and recomputed local factors. We express this condition as a small CUDA Tile kernel-generation interface and implement it in CutileReduce. The same interface generates forward and backward kernels for cross entropy, attention, and a non-commutative affine attention variant, reducing memory while remaining competitive with PyTorch baselines.

1 Notation

Types and Products: We write $a : A$ to denote that a is of type A. Named product types are written as $\{a : A, b : B\}$, with corresponding instantiated terms written as $\{a : v, b : w\}$.

Functions and Application: Function arrows associate to the right: $A \to B \to C \equiv A \to (B \to C)$. Standard parentheses $f(x)$ denote function application on primal parameters or state. Conversely, square brackets $f[v]$ are reserved for operations acting on dual objects (gradients, cotangents, or accumulations). A parameterized map acting on a dual object v is written as $f(x)[v]$.

Composition and Multiplication: Symbol $\circ$ denotes function composition: $(f \circ g)(x) = f(g(x))$. Matrix multiplication is denoted using standard juxtaposition (e.g., AB). $a \cdot \mathbf{x}$ denotes scalar multiplication. When applicable, this also applies to records: $\alpha \cdot \{a : v, b : w\} = \{a : \alpha \cdot v, b : \alpha \cdot w\}$

Derivatives, Pullbacks and Cotangents: Inspired by Elliott (2018), $\frac{\mathrm{d}\, a}{\mathrm{d}\, b} : A^* \to B^*$ denotes the pullback linear map (the transpose Jacobian) that transforms cotangents (gradients) of $a : A$ into cotangents of $b : B$. Accordingly, the chain rule is written as the composition of these linear maps: $\frac{\mathrm{d}\, c}{\mathrm{d}\, a} = \frac{\mathrm{d}\, b}{\mathrm{d}\, a} \circ \frac{\mathrm{d}\, c}{\mathrm{d}\, b}$. For a scalar loss ℓ, $\bar{x} : X^*$ denotes the cotangent resulting from the pullback $\frac{\mathrm{d}\, \ell}{\mathrm{d}\, x}[1]$.

2 Introduction

Modern efficient neural kernels often avoid materializing large logical interaction tensors, such as attention logits, vocabulary logits, and pairwise score matrices. Existing examples, including FlashAttention and Cut Cross-Entropy, are typically derived and implemented as separate engineering artifacts.

This paper argues that a useful subset of these kernels share a common structure: they are monoidal folds whose backward passes satisfy a local gradient rule. The rule is not a replacement

for automatic differentiation in general; it is a sufficient condition under which the backward pass can be computed by recomputing local factors and using only compact fold state.

Contributions.

- We state the Local Gradient Property for commutative and general monoidal folds.
- We introduce execution carriers as stable and hardware-friendly representations of semantic monoids.
- We formulate the finalize-embed factorization as a compiler-facing backward interface.
- We implement this interface in CutileReduce, a CUDA Tile generator for tiled fold kernels.
- We evaluate the implementation on cross entropy, attention, and affine LWS attention.

3 Map Fold Functions

A broad class of functions $\mathbf{y} = f(\mathbf{x})$ can be decomposed as

$$\begin{aligned} \mathbf{l}_r &= \mathrm{map}(\mathbf{x}, r) \\ \mathbf{g} &= \mathrm{fold}_r(\mathbf{l}_r) \\ &= \mathbf{l}_0 \odot \mathbf{l}_1 \odot \cdots \\ \mathbf{y} &= \mathrm{out}(\mathbf{g}) \end{aligned}$$

where each local factor $\mathbf{l}_r$ belongs to a monoid $(S, \odot)$. A common approach to reduce space complexity is map-fold fusion: Instead of realizing all intermediate local factors $\mathbf{l}$, factors are combined in place, pushing space complexity from $O(R)$ to $O(1)$.

In the context of machine learning we are not only interested in computing the forward pass $\mathbf{y} \leftarrow f(\mathbf{x})$, but also the backward pass $\frac{\mathrm{d}\,\mathbf{y}}{\mathrm{d}\,\mathbf{x}}\,[\bar{\mathbf{y}}]$. In the next section we introduce the Local Gradient Property, a sufficient condition for when map fold functions over monoids can be made memory efficient.

4 The Local Gradient Property

4.1 Commutative Folds

Definition 1 (Commutative Local Gradient Property)**.** *Let $(S, \odot)$ be a commutative monoid on a finite-dimensional smooth space. We say that S satisfies the commutative Local Gradient Property if there exists a function*

$$D : S \times S \to S^* \to S^*$$

such that, for all $a, b \in S$,

$$\frac{\mathrm{d}\,a \odot b}{\mathrm{d}\,a} = D(a \odot b, a).$$

Proposition 1 (Local gradients for commutative folds)**.** *If $(S, \odot)$ satisfies the commutative Local Gradient Property and $g = \bigodot_i l_i$, then*

$$\frac{\mathrm{d}\,g}{\mathrm{d}\,l_i} = D(g, l_i).$$

Proof. By the monoid laws and commutativity, there exists b such that $g = l_i \odot b$. From there the result follows directly from definition 1. □

Example: LogWeightedSum As an illustrative example, we introduce the LogWeightedSum monoid, which forms the basis for attention, cross-entropy, and other information theoretic measures:

$$\begin{aligned} LWS &\coloneqq \{z : \mathbb{R},\, \mu : \mathbb{R}^D\} \\ a \odot_{LWS} b &= \{z : \text{logaddexp}(a_z,\, b_z),\, a_\mu \cdot \exp(a_z - z) + b_\mu \cdot \exp(b_z - z)\} \end{aligned}$$

The LWS monoid is commutative, and satisfies the commutative LGP:

$$\begin{aligned} \frac{\mathrm{d}\, a \odot b}{\mathrm{d}\, a}[\Delta] &= \exp(a_z - (a \odot b)_z) \cdot \{z : \Delta_z + \langle \Delta_\mu,\, a_\mu - (a \odot b)_\mu \rangle,\, \mu : \Delta_\mu\} \\ &\Downarrow \\ D(g, l)[\Delta] &= \exp(l_z - g_z) \cdot \{z : \Delta_z + \langle \Delta_\mu,\, l_\mu - g_\mu \rangle,\, \mu : \Delta_\mu\} \end{aligned}$$

4.2 General Folds

Definition 2 (General Local Gradient Property)**.** *Let $(S, \odot)$ be a monoid. We say that S satisfies the general Local Gradient Property if there exists a function*

$$D : S \times S \times S \to S^* \to S^*$$

such that

$$\frac{\mathrm{d}\, a \odot l \odot b}{\mathrm{d}\, l} = D(a \odot l \odot b, a \odot l, l).$$

Proposition 2 (Local gradients for general folds)**.** *Let $g = l_1 \odot \cdots \odot l_n$ and $p_i = l_1 \odot \cdots \odot l_i$ for $i \leq n$. If $(S, \odot)$ satisfies the general Local Gradient Property, then*

$$\frac{\mathrm{d}\, g}{\mathrm{d}\, l_i} = D(g, p_i, l_i).$$

Proof. By the monoid laws, there exists a and b such that $p_i = a \odot l_i$ and $g = a \odot l_i \odot b$. From there the result follows directly from definition 2. □

5 The finalize-embed factorization

The Local Gradient Property states that local gradients can be computed from a global state, a local factor, and, for general folds, prefix state. Broadcasting both global state and its gradient to all workers is, however, unnecessarily wasteful. To combat this we introduce the finalize-embed factorization:

$$\begin{aligned} &\text{embed} : S \to S^* \to A \\ &\text{finalize} : S \to A \to S^* \quad \text{(commutative)} \\ &\text{finalize} : S \times S \to A \to S^* \quad \text{(general)} \end{aligned}$$

$$D(g, p, l) = \text{finalize}(p, l) \circ \text{embed}(g) \tag{1}$$

Where A acts as a compact gradient representation. For folds, and the purpose of this paper, the benefit of this factorization is that it decreases total work and read traffic by precomputing A and broadcasting that, as opposed to broadcasting both S and S^* and computing D directly. However, in appendix B.1 we sketch an extension of this factorization to enable efficient scans.

Example: Factorization of D_{LWS} The LGP-style gradient function for the LogWeightedSum monoid is:

$$D(g, l)[\Delta] = \exp(l_z - g_z) \cdot \{z : \Delta_z + \langle \Delta_\mu,\, l_\mu - g_\mu \rangle,\, \mu : \Delta_\mu\}$$

By isolating terms dependent on g, we get the following:

$$\begin{aligned} \text{embed}(g)[\Delta] &= \exp(-g_z) \cdot \{z : \Delta_z - \langle \Delta_\mu,\, g_\mu \rangle,\, \mu : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z) \cdot \{z : \eta_z + \langle \eta_\mu,\, l_\mu \rangle,\, \mu : \eta_\mu\} \end{aligned}$$

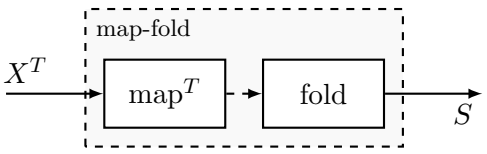
Since $\exp(-g_z)$ and $\exp(l_z)$ can lead to numerical instability, the alternative compact gradient representation is preferable:

$$\begin{aligned} A &\coloneqq \left\{z : \mathbb{R},\, s : \mathbb{R},\, v : \mathbb{R}^D\right\} \\ \text{embed}(g)[\Delta] &= \{z : -g_z,\, s : \Delta_z - \langle \Delta_\mu,\, g_\mu \rangle,\, v : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z + \eta_z) \cdot \{z : \eta_s + \langle \eta_v,\, l_\mu \rangle,\, \mu : \eta_v\} \end{aligned}$$

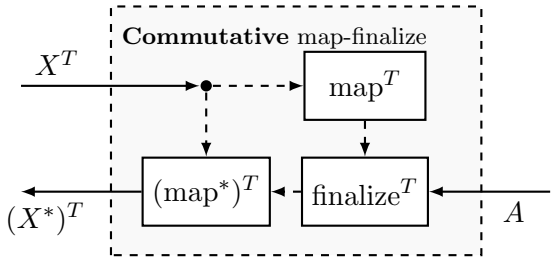
6 Implementation sketch

With the algebraic machinery above, we can now sketch our implementation of tiled forward and backward computation for map fold functions over monoids satisfying the Local Gradient Property with a finalize-embed factorization. In the sketch, we let X^F denote a global interaction view over tensor bundles. As an example, for attention X would correspond to a Query, Key, and Value vector tuple, realizing

6.1 Commutative Tiled Map Fold Fusion



(a) Schematic for tiled, map-fold.



(b) Schematic for tiled, commutative map-finalize.

Figure 1: The forward backward interface for commutative tiled map fold fusion: The implementation exposes callbacks for batched, tile-local map-fold (a) and map-finalize (b), and batched $\odot$ operations.

Forward pass The forward pass consists of tiling the global interaction view X^F into a Program, Loop, and Tile-partitioned view $X^{P \times L \times T}$. This view is fed into a partial map-fold stage, with partial results aggregated in a fold stage to produce the final output. In figure 2, map-fold$_L^P$ denotes a fused map-fold function parallelized over P programs, with each program looping over L tiles of size T,

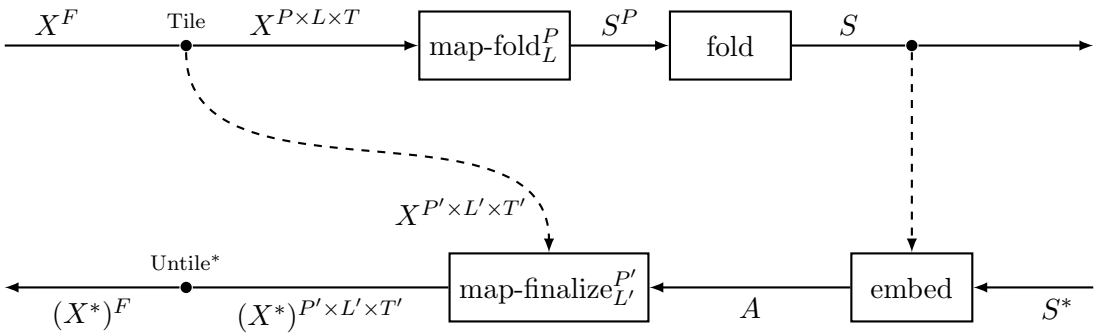


Figure 2: Forward and backward passes of map fold functions over commutative $(S, \odot)$.

producing output of type S^P:

$$\text{map-fold}_L^P : X^{P\times L\times T} \to S^P$$
$$\text{map-fold}_L^P(\mathbf{x})_p = \bigodot_{l\in L} (\text{map-fold}(\mathbf{x}_{p,l}))$$

The P independent partial results are written to memory and fed to the separate fold stage to produce the final result S. In the case where one program is responsible for the full fold (i.e. $P = 1$), the fold stage is omitted[1]. Only the input (X^F) and final result (S) are persisted for the backward pass.

Backward pass The backward pass uses the finalize-embed-factorization to first embed the output and output gradient into compact representation A. This is then broadcast to the P'-parallel L'-looping map-finalize$_{L'}^{P'}$-function. For input buffers that do not depend on the loop axis F, gradients are aggregated in place and added to global input buffer gradient on program end using atomic add, whereas input buffers that depend on F are added to global input buffers using atomic add inside the loop.

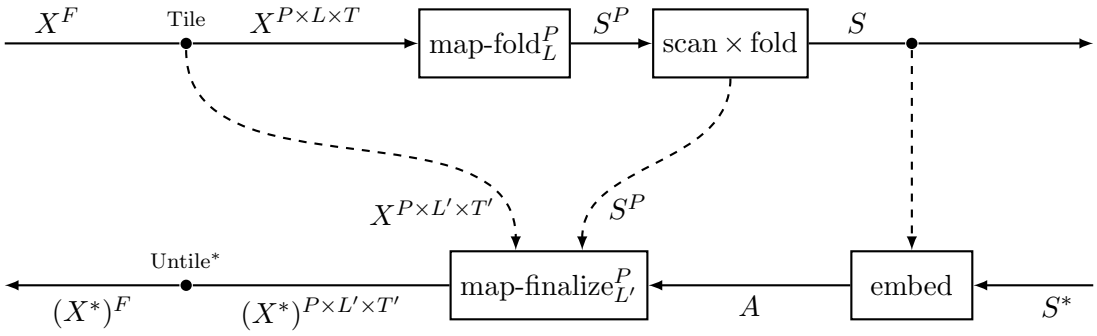


Figure 3: Forward and backward passes of map fold functions over non-commutative $(S, \odot)$.

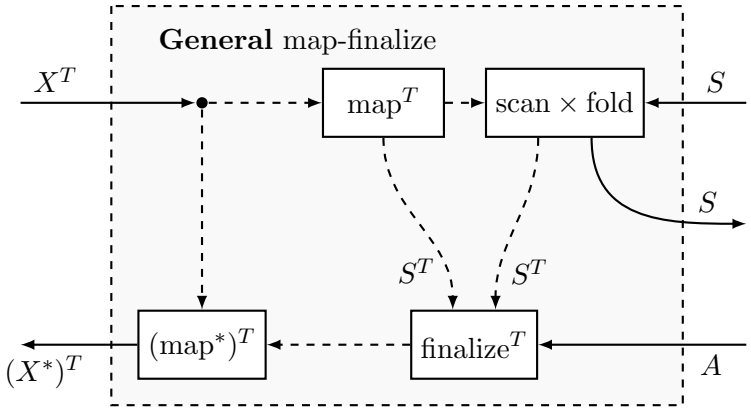


Figure 4

6.2 General Tiled Map Fold Fusion

Forward pass For a general monoid, the partial summaries cannot be combined in an arbitrary order. The partial map-fold stage therefore produces an ordered sequence $\mathbf{q} \in S^P$, where $\mathbf{q}_p$ summarizes the fold tiles assigned to program p. The second stage performs an ordered exclusive scan over these summaries, and also returns the total fold:

$$\begin{aligned} \text{scan} \times \text{fold} &: S^P \to S^P \times S \\ (\text{scan} \times \text{fold})(\mathbf{q}) &= (\mathbf{s}, g), \\ \mathbf{s}_p = \bigodot_{j<p} \mathbf{q}_j, &\qquad g = \bigodot_{j} \mathbf{q}_j. \end{aligned}$$

The value s_p is the left-boundary state for program p. It is exactly the state needed to resume the ordered fold inside that program. The implementation may either persist these boundary states for the backward pass or recompute them from the partial summaries.

Backward pass The backward pass first applies embed to the global output state and its cotangent, producing the compact gradient accumulator A. Each backward program receives this same accumulator together with its left-boundary state $\mathbf{s}_p$. It then recomputes the local factors for its assigned tiles and threads a running prefix through the loop:

$$\mathbf{p}_{p,0} = \mathbf{s}_p, \qquad \mathbf{p}_{p,l+1} = \mathbf{p}_{p,l} \odot \mathbf{l}_{p,l}.$$

For each tile, map-finalize consumes the current prefix, the recomputed local factor, and the embedded gradient accumulator, and emits gradients for the input tile. Thus the only additional state required by the general case, compared to the commutative case, is the ordered prefix boundary and the intra-program prefix threaded through the recomputation loop.

6.3 Tiled Backward Consequence

Theorem 1 (Memory-efficient tiled fold backpropagation)**.** *Consider a map fold function over a monoid that satisfies the appropriate Local Gradient Property and has a* finalize-embed *factorization. Then the reverse-mode gradient of the map fold function can be computed without materializing*

[1]Note that this often is a valid strategy: With large enough batch dimension, parallelizing over batches can achieve sufficient occupancy.

the full logical factor tensor. For a problem over batch axis B and fold axis F, both forward and backward pass can be computed using only $O(B \times P)$ space, where P is a scheduling parameter (possibly 1), as opposed to the batched solution using $O(B \times F)$ space.

Remark 1. *The theorem gives a storage guarantee, not a universal speed guarantee. The backward pass recomputes local factors and applies the local* finalize *rule. For common tensor-contraction maps this is the same asymptotic traversal class as the corresponding forward computation, but with constants determined by the specific gradient contractions.*

7 Execution Carriers

Semantic monoids are not always the representation used in an efficient kernel. For example, log-sum-exp is often computed using a maximum and a rescaled exponential sum. We model this using the notion of an execution carrier.

Definition 3 (Execution carrier)**.** *Let $(S, \odot_S)$ be a semantic monoid. An execution carrier for S is a monoid $(E, \odot_E)$ together with a surjective monoid homomorphism $\Psi : E \to S$, with right inverse $\Phi : S \to E$:*

$$\Psi(e_1 \odot_E e_2) = \Psi(e_1) \odot_S \Psi(e_2) \tag{2}$$

$$\Psi \circ \Phi = \mathrm{id}_S \tag{3}$$

From definition 3 it follows that $\bigodot_i l_i = \Psi(\bigodot_i \Phi(l_i))$. This identity allows us to compute folds or scans in an execution carrier E, and utilize the LGP-backwards pass for reconstructed semantic primal states in S directly, computed via Ψ, rather than threading the gradient pass through the execution carrier.

Example: Fused Max-Trick as an Execution Carrier for LWS When computing $\mathrm{logaddexp}(a, b)$, a standard trick to improve numerical stability is the max-trick:

$$\mathrm{logaddexp}(a, b) = h + \log(1 + \exp(l - h)) \tag{4}$$

$$\text{where } (h, l) = \begin{cases} (a, b) & a > b \\ (b, a) & a \leq b \end{cases} \tag{5}$$

Unrolling this, we can build an execution carrier for LogWeightedSum that tracks the maximum m and rescales the sum of exponents e and paylods u accordingly:

$$\begin{aligned} E_{LWS} &\coloneqq \{m : \mathbb{R},\, e : \mathbb{R},\, u : \mathbb{R}^D\} \\ a \odot_{E_{LWS}} b &= \{m : h_m,\, e : h_e + \alpha \cdot l_e,\, u : h_u + \alpha \cdot l_u\} \\ \text{where } (h, l) &= \begin{cases} (a, b) & a_m > b_m \\ (b, a) & a_m \leq b_m \end{cases} \\ \alpha &= \exp(l_m - h_m) \\ \Phi(x) &= \{m : x_z,\, e : 1,\, u : x_\mu\} \\ \Psi(y) &= \left\{ z : y_m + \log(y_e),\, \mu : \frac{y_u}{y_e} \right\} \end{aligned}$$

This algebraic structure corresponds to the online scaling and denominator tracking used in the forward pass of FlashAttention (Dao et al., 2022).

8 CutileReduce

CutileReduce is a CUDA Tile implementation of the fold interface sketched above. A user provides a fold specification and a collection of callbacks:

```
spec = make_fold_spec(...)
functions = fold_functions(
    map_fold,
    combine,
    to_semantic,
    to_output,
    embed=embed,
    map_finalize=map_finalize,
)
op = FoldOperator(spec, functions)
plan = op.tune(sizes, hardware=rtx5080)
fn = op.build(plan)
```

$$\text{map-fold} : X^T \to E \tag{6}$$

$$\text{combine} : E \times E \to E \qquad (\text{combine} = \odot_E) \tag{7}$$

$$\text{to-semantic} : E \to S \qquad (\text{to-semantic} = \Psi) \tag{8}$$

$$\text{to-output} : S \to Y \tag{9}$$

$$\text{embed} : S \times S^* \to A \tag{10}$$

$$\text{map-finalize} : X^T \times A \to (X^*)^T \qquad (\textit{Commutative}) \tag{11}$$

$$\text{map-finalize} : X^T \times E \times A \to \left((X^*)^T, E\right) \qquad (\textit{General}) \tag{12}$$

$$\tag{13}$$

The fold interface does not require the state returned by map-fold to be the semantic monoid state itself. Instead, the implementation folds whatever carrier type E is returned by map-fold, using the user-provided combine : $E \times E \to E$. If to-semantic = id, then $E = S$ and the kernel folds directly in the semantic monoid. Otherwise, to-semantic : $E \to S$ is interpreted as the projection from an execution carrier to the semantic state. Correctness is preserved whenever $(E, \odot_E)$ is an execution carrier for $(S, \odot_S)$, according to definition 3.

Thus CutileReduce is agnostic to whether the folded state is semantic or an implementation-specific stable representation; it only requires the callbacks to expose a monoid operation on the chosen carrier and a final conversion to the declared output state.

The concrete implementation follows the staged structure in Figures 2 and 3. A full fold is represented as a single map-fold stage. When the fold axis is split across multiple programs, CutileReduce emits a partial map-fold stage followed by either a commutative fold stage or, in the general case, an ordered scan × fold stage that also produces prefix carriers for backward recomputation. The backward pass is generated as a recomputation kernel using embed and map-finalize; checkpointed general plans reuse the saved prefix carriers from the ordered scan.

8.1 Tuning and Persistence

CutileReduce performs an analytical sweep over legal tile and partition choices, then empirically times selected stage candidates. Selected plans can be saved as JSON and reloaded without re-

running the sweep. CUDA Tile remains responsible for kernel compilation and low-level code generation.

8.2 Implementation Limitations

The current prototype supports a single fold/loop axis. Inner axes that are neither batch nor fold axes must fit within one program tile. The tuning search does not yet cover CUDA Tile compiler hints, richer layout choices, or explicit pipeline policies.

9 Transport

The Local Gradient Property and the finalize-embed factorization need not be derived from scratch for every monoid. They are preserved by smooth injective monoid homomorphisms. Thus, if a semantic monoid S embeds into a monoid T whose backward rule is known, the rule for S can be obtained by transporting cotangents through the embedding and its inverse.

Two base cases are particularly useful. General non-commutative monoids can often be embedded into $GL_N(\mathbb{R})$, where products are matrix products and prefixes naturally appear in the local gradient rule. Commutative monoids often embed into $(\mathbb{R}^N, +)$, where the local rule is trivial and all structure is carried by the coordinate transform. The unstable log-weighted-sum factorization in Section 5, for example, arises by transporting LWS through total mass coordinates. We give the transport statements and derivations in Appendix **??**.

10 Instances

10.1 One-hot Cross Entropy

For logits $z_{b,v} = \langle c_b, t_v \rangle$ and target index τ_b, the loss is

$$\ell_b = \log \sum_v \exp z_{b,v} - z_{b,\tau_b}.$$

The carrier tracks the log-sum-exp state and target logit. The logical $B \times V$ logits matrix is never materialized.

10.2 Attention

Scaled dot-product attention can be expressed using a log-weighted-sum carrier:

$$\mathrm{Attn}(Q, K, V)_i = \left(\bigodot_j \{ z : \langle Q_i, K_j \rangle,\ \mu : V_j \} \right)_\mu .$$

The execution carrier implements the stable online softmax representation.

10.3 Affine LWS Attention

Affine LWS attention is a non-commutative fold instance. It augments the attention carrier with an order-dependent affine/log-bias component, requiring prefix state in the general Local Gradient Property. This example demonstrates that the interface applies beyond commutative reductions that were already covered by existing specialized kernels.

11 Evaluation

11.1 Experimental Setup

Report GPU, CUDA Tile version, PyTorch version, dtype, benchmark method, warmup, timing duration, and whether tuning time is included.

11.2 Correctness

Kernel	Output MAE	Grad 1 MAE	Grad 2 MAE	Grad 3 MAE
Cross entropy	TODO	TODO	TODO	–
Attention	TODO	TODO	TODO	TODO
Affine LWS attention	TODO	TODO	TODO	TODO

Table 1: Correctness against PyTorch reference implementations.

11.3 Timing

Kernel	Mode	CutileReduce	Baseline
Cross entropy	Forward	TODO	TODO
Cross entropy	Forward + backward	TODO	TODO
Attention	Forward	TODO	TODO
Attention	Forward + backward	TODO	TODO
Affine LWS attention	Forward	TODO	TODO
Affine LWS attention	Forward + backward	TODO	TODO

Table 2: Wall-clock benchmark results.

11.4 Memory

Kernel	Mode	CutileReduce	Baseline
Cross entropy	Forward	TODO	TODO
Cross entropy	Forward + backward	TODO	TODO
Attention	Forward	TODO	TODO
Attention	Forward + backward	TODO	TODO
Affine LWS attention	Forward	TODO	TODO
Affine LWS attention	Forward + backward	TODO	TODO

Table 3: Incremental peak PyTorch-managed CUDA allocation.

12 Related Work

Memory-efficient neural kernels. Discuss FlashAttention, FlashAttention-2, Cut Cross-Entropy, Liger kernels, and other fused training kernels.

Automatic differentiation of parallel primitives. Discuss AD for reductions and scans in array languages, DrJAX, Futhark, and related differentiable MapReduce systems.

Symbolic and lazy reductions. Discuss KeOps and related symbolic reduction systems.

Kernel DSLs and code generation. Discuss Triton, CuTe DSL, TileLang, CUDA Tile, and how CutileReduce differs by placing an algebraic fold interface above tile-level kernel programming.

Checkpointing and recomputation. Discuss activation checkpointing, reversible methods, and recomputation-based memory reductions.

13 Limitations

The Local Gradient Property is sufficient, not necessary. Some useful operations may admit memory-efficient backward passes outside this framework. Conversely, satisfying the property does not imply that recomputation is always a good performance tradeoff.

General non-commutative folds require prefix state or prefix recomputation. The current implementation is NVIDIA CUDA Tile specific, supports only one fold axis, and has a limited tuning space. First-class scan and scan-fold APIs are left to future work.

14 Conclusion

Monoidal folds provide a compact way to describe a class of memory-efficient neural kernels. The Local Gradient Property and the finalize-embed factorization turn this observation into a compiler-facing backward interface. CutileReduce demonstrates that the interface can generate practical CUDA Tile forward and backward kernels for known and new fold-style operators.

References

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Re, C. (2022). Flashattention: Fast and memory-efficient exact attention with IO-awareness. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K., editors, *Advances in Neural Information Processing Systems*.

Elliott, C. (2018). The simple essence of automatic differentiation. *Proc. ACM Program. Lang.*, 2(ICFP).

A Transport

Definition 4 (Transport)**.** *For monoids $(S, \odot)$ and $(T, \otimes)$, we say that S transports to T if there exists an injective, smooth homomorphism $\phi : (S, \odot) \hookrightarrow (T, \otimes)$, with a smooth left inverse on its image:*

$$\phi(s_1 \odot_S s_2) = \phi(s_1) \odot_T \phi(s_2) \tag{14}$$
$$\psi \circ \phi = \mathrm{id}_S \tag{15}$$

Proposition 3 (Transport of the Local Gradient Property)**.** *If S transports to T, and T satisfies a Local Gradient Property, then so does S with*

$$\textit{General LGP:} \qquad D_S(g,p,l) = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l} \circ D_T(\phi(g),\phi(p),\phi(l)) \circ \frac{\mathrm{d}\,\psi(\phi(g))}{\mathrm{d}\,\phi(g)} \tag{16}$$

$$\textit{Commutative LGP:} \qquad D_S(g,l) = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l} \circ D_T(\phi(g),\phi(l)) \circ \frac{\mathrm{d}\,\psi(\phi(g))}{\mathrm{d}\,\phi(g)} \tag{17}$$

Proof. The result follows directly from the properties of monoid homomorphisms and the chain rule. □

Proposition 4 (Transport of the finalize-embed factorization)**.** *If S transports to T, and T satisfies a Local Gradient Property with a* finalize-embed *factorization, then so does S with:*

$$\textit{General LGP:} \qquad \mathrm{finalize}_S(p,l) = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l} \circ \mathrm{finalize}_T(\phi(p),\phi(l))$$

$$\textit{Commutative LGP:} \qquad \mathrm{finalize}_S(l) = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l} \circ \mathrm{finalize}_T(\phi(l))$$
$$\mathrm{embed}_S(g) = \mathrm{embed}_T(\phi(g)) \circ \frac{\mathrm{d}\,\psi(\phi(g))}{\mathrm{d}\,\phi(g)}$$

Proof. The results follows from substitution of D_T by $\mathrm{finalize}_T \circ \mathrm{embed}_T$ in Proposition 3 and grouping the composition by associativity. □

A.1 $GL_N(\mathbb{R})$ satisfies the General LGP

For GL_N we have that

$$D(g,p,l)[\Delta] = (pl^{-1})^\intercal \Delta (p^{-1}g)^\intercal \tag{18}$$
$$\mathrm{finalize}_{GL_N(\mathbb{R})}(p,l)[\eta] = l^{-\intercal} p^\intercal \, \eta \, p^{-\intercal} \tag{19}$$
$$\mathrm{embed}_{GL_N(\mathbb{R})}(g)[\Delta] = \Delta g^\intercal \tag{20}$$

By Propositions 3 and 4, any monoid S with transport $(S,\odot) \hookrightarrow GL_N(\mathbb{R})$ satisfies the General LGP with the following concrete instances of finalize and embed:

$$\mathrm{finalize}_S(p,l)[\eta] = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l}\left[\phi(l)^{-\intercal}\phi(p)^\intercal \, \eta \, \phi(p)^{-\intercal}\right] \tag{21}$$
$$\mathrm{embed}_S(g)[\Delta] = \frac{\mathrm{d}\,\psi(\phi(g))}{\mathrm{d}\,\phi(g)}\,[\Delta]\;\phi(g)^\intercal \tag{22}$$

A.2 $(\mathbb{R}^N, +)$ satisfies the Commutative LGP

A simpler and quite common special case of $GL_N(\mathbb{R})$ is $(\mathbb{R}^N, +)$. This satisfies the Commutative LGP:

$$\begin{aligned} D_{(\mathbb{R}^N,+)}(a \odot b, a) &= \mathrm{id} \\ \mathrm{finalize}_{(\mathbb{R}^N,+)}(l) &= \mathrm{id} \\ \mathrm{embed}_{(\mathbb{R}^N,+)}(g) &= \mathrm{id} \end{aligned}$$

By Propositions 3 and 4, any monoid S with transport $(S, \odot) \hookrightarrow (\mathbb{R}^N, +)$ satisfies the Commutative LGP with the following concrete instances of finalize and embed:

$$\mathrm{finalize}_S(l) = \frac{\mathrm{d}\,\phi(l)}{\mathrm{d}\,l} \tag{23}$$

$$\mathrm{embed}_S(g) = \frac{\mathrm{d}\,\psi(\phi(g))}{\mathrm{d}\,\phi(g)} \tag{24}$$

Example: *LWS* is transportable to $(\mathbb{R}^N, +)$ LogWeightedSum is transportable to $(\mathbb{R}^N, +)$ by transforming the log-space weight to the corresponding total weight by exponentiation, and by transforming the mean to the total sum by multiplication by the total weight, along with the inverse operation:

$$\begin{aligned} \phi(a) &= \{w : \exp(a_z),\ s : w \cdot a_\mu\} \\ \psi(x) &= \left\{z : \log(x_w),\ \mu : \frac{1}{x_w} \cdot x_s\right\} \end{aligned}$$

B Scans

B.1 The finalize-embed factorization for scans

For scans, the interface proposed so far breaks down, since $\bar{l}_i$ require access to all future prefixes p_j and their gradients $\bar{p}_j$:

$$\bar{l}_i = \sum_{j \geq i} D(p_j, p_i, l_i)\,[\bar{p}_j] \tag{25}$$

To address this limitation, we extend on the finalize-embed factorization, requiring that $(A, \oplus)$ is a (partial) semigroup, such that

$$\sum_{j \geq i} D(p_j, p_i, l_i)\,[\bar{p}_j] = \mathrm{finalize}(p, l)\left[\bigoplus_{j \geq i} \mathrm{embed}(p_j)\,[\bar{p}_j]\right] \tag{26}$$

From which it follows that:

$$\bar{l}_i = \mathrm{finalize}(p_i, l_i)\,[\eta_i] \qquad \eta_i = \bigoplus_{j \geq i} \mathrm{embed}(p_j)\,[\bar{p}_j] \tag{27}$$

The factorization transport property (Proposition 4) remains unchanged under this stricter notion of the finalize-embed factorization.

Example: Gradient accumulator for LWS *is* LWS In 5 we gave an unstable finalize-embed factorization of LWS as:

$$\begin{aligned}\text{embed}(g)[\Delta] &= \exp(-g_z) \cdot \{z : \Delta_z - \langle \Delta_\mu,\, g_\mu \rangle,\, \mu : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z) \cdot \{z : \eta_z + \langle \eta_\mu,\, l_\mu \rangle,\, \mu : \eta_\mu\}\end{aligned}$$

That factorization maps to the additive gradient accumulator $(\{z : \mathbb{R},\, \mu : \mathbb{R}^D\}, +)$. This illustrates an interesting duality: The gradient accumulator for a LogWeightedSum *is* a LogWeightedSum, giving the following alterbative gradient accumulator: [2]

$$\begin{aligned}A &\coloneqq LWS(\{s : \mathbb{R},\, v : \mathbb{R}^D\}) \\ \text{embed}(g)[\Delta] &= \{z : -g_z,\, s : \Delta_z - \langle \Delta_\mu,\, g_\mu \rangle,\, v : \Delta_\mu\} \\ \text{finalize}(l)[\eta] &= \exp(l_z + \eta_z) \cdot \{z : \eta_s + \langle \eta_v,\, l_\mu \rangle,\, \mu : \eta_v\}\end{aligned}$$

C Instance Derivations

Provide full derivations for log-sum-exp, log-weighted sum, one-hot cross entropy, attention, and affine LWS attention.

D Implementation Details

Describe stage domains, buffer roles, generated CUDA Tile programs, tuning search spaces, and plan persistence.

E Additional Experiments

Include shape sweeps, failure cases, tuning costs, extra baselines, and saved plan metadata.

[2] Here, $LWS(x)$ denotes a LWS-monoid with a weighted sum of type x. That is, for $LWS(\{s : \mathbb{R},\, v : \mathbb{R}^D\})$

$$a \odot b = \begin{Bmatrix} z : \text{logaddexp}(a_z, b_z) \\ s : \exp(a_z - z) \cdot a_s + \exp(b_z - z) \cdot b_s \\ v : \exp(a_z - z) \cdot a_v + \exp(b_z - z) \cdot b_v \end{Bmatrix} \tag{28}$$